

Sprawozdanie Etap 4 - Neo4j

Zaawansowane systemy baz danych

Krzysztof Dąbrowski (293101)

GR6 (Poniedziałek 8:15)

Spis treści

1	Zbiór danych	1
2	Instalacja Neo4j i wczytanie danych	2
2.1	Instalacja	2
2.2	Wczytanie danych	4
3	Dołączenie drugiego zbioru danych	6
3.1	Wczytanie danych o bitwach	7
3.2	Wykorzystanie rozszerzonych danych	8
3.3	Ręczne tworzenie węzłów	10
4	Optymalizacja zapytań	10
4.1	Indeksy	10
4.2	Optymalizacja struktury grafu	12
5	Dane przestrzenne	12
6	Procedury	13
7	Analiza produkcyjna	14
8	APOC	15
9	Wnioski	16
	Źródła	17
	Zastosowane skróty	20

Spis rycin

1	Bazowy schemat danych Game of Graphs. Źródło: Repozytorium Game of Graphs [1]	2
2	Podgląd danych Game of Graphs. Źródło: Repozytorium Game of Graphs [1]	3
3	Wizualizacja wczytanych danych z <i>Api of Ice And Fire</i> [7]	6
4	Gałęzie rodu Lannisterów	9
5	Zapytanie o gałęzie rodu Lannisterów z i bez indeksu	11
6	Wizualizacje wybranych ciekawych procedur <i>APOC</i>	16

Spis tabel

1	Liczba węzłów po wczytaniu danych	5
2	Liczba relacji po wczytaniu danych	6
3	Relacje tworzone w skrypcie <code>import__battles.cypher</code>	7
4	Liczba relacji związanych z bitwami	8
5	Gałęzie rodu Lannisterów	9
6	Statystyki spójnych podgrafów w bazie	14

Spis wykazów

1	Nieudane utworzenie bazy w Cypher	4
2	Relacje Eddarda Starka	8

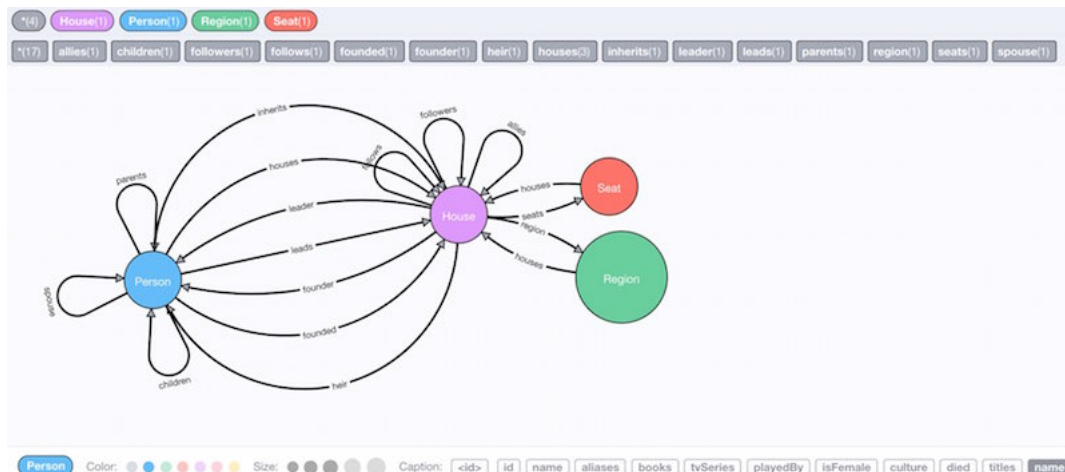
Kod projektu

Kod projektu i sprawozdań jest dostępny na GitHub [BoardBase](#) 

1 Zbiór danych

Wybrałem zbiór danych *Game of Graphs* [1] zawierający informacje o postaciach, rodach, rejonach i siedzibach rodów z serii książek *Pieśń Lodu i Ognia* (Gra o Tron) autorstwa George'a R.R. Martina [2] [3] [4] [5] [6]. Zdecydowałem się na ten zbiór, ponieważ jestem fanem fantastyki. Sądzę, że ciekawe będzie przeanalizowanie go.

Zbiór *Game of Graphs* [1] został przygotowany na podstawie danych z projektu *An API of Ice And Fire* [7]. W zbiorze bżowo występują 4 rodzaje węzłów oraz 17 rodzajów relacji widocznych na Rysunek 1. Bazowy zbiór zawiera 2122 węzłów i 3233 relacji widocznych na podglądzie Rysunek 2.



Rysunek 1: Bazowy schemat danych Game of Graphs. Źródło: Repozytorium Game of Graphs [1]

Ponieważ bazowy zbiór *Game of Graphs* [1] zawiera tylko 4 rodzaje węzłów, dodam do niego typ węzła *Book* reprezentujący książki z serii. Projekt *An API of Ice And Fire* [7] dostarcza endpoint pozwalający na pobranie informacji o książkach. Nie został on jednak wykorzystany w bazowym zbiorze *Game of Graphs* [1].

2 Instalacja Neo4j i wczytanie danych

Przygotowanie bazy danych i wczytanie wstępnych danych.

2.1 Instalacja

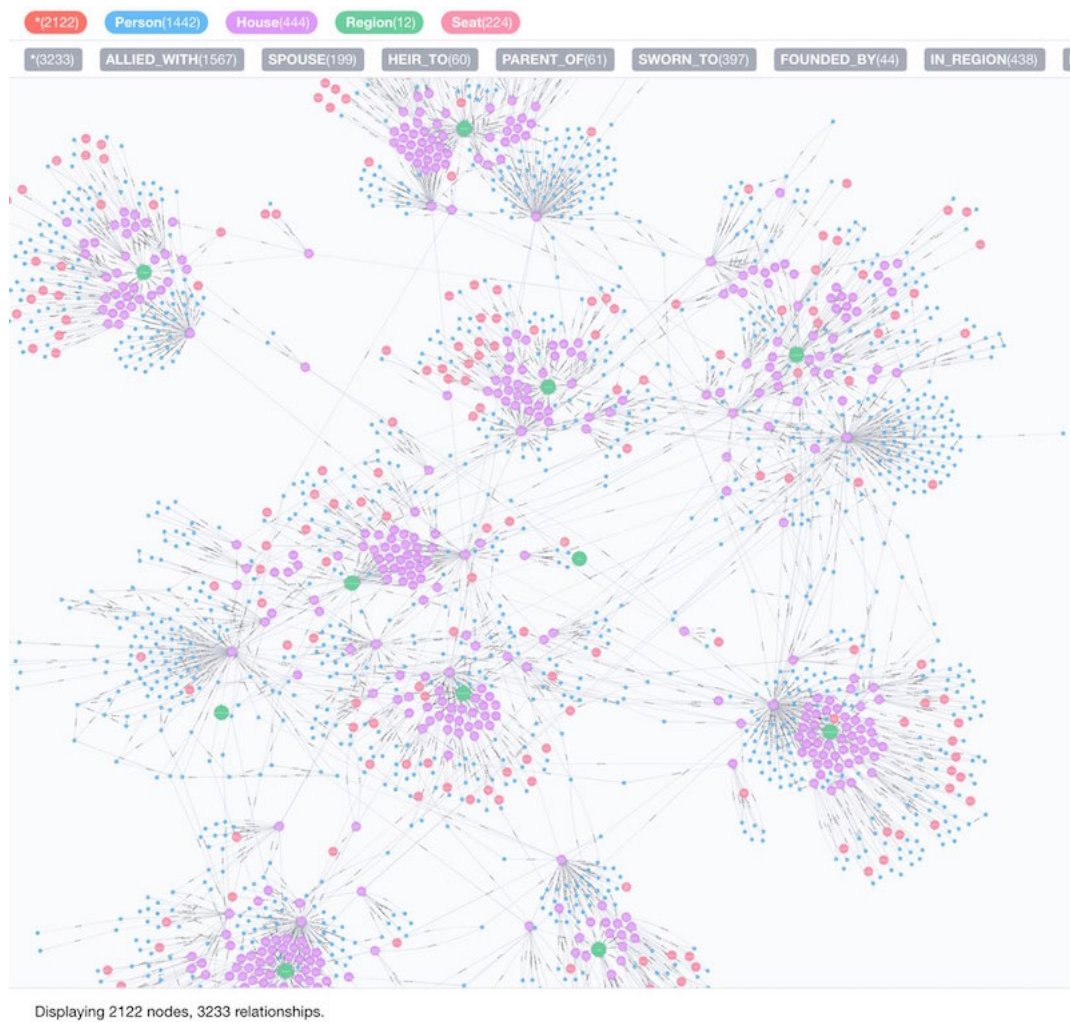
Wybrałem wersję Neo4j *Community Edition*, ponieważ jest ona darmowa [8]. Nie potrzebuję funkcji dostępnych w wersji *Enterprise Edition* [8].

Zdecydowałem się na użycie Docker'a, w celu zapewnienia replikowalności mojego środowiska. Skonfigurowałem Neo4j jako kontener Docker w pliku `docker-compose.yml`, bazując na instrukcjach z dokumentacji Neo4j [9] [10].

Żeby skonfigurować Neo4j zdecydowałem się na pobranie domyślnych plików konfiguracyjnych, modyfikację ich i podmontowanie do kontenera.

Bazową konfigurację pobrałem poleceniem `docker run --rm --volume=./neo4j/conf:/conf neo4j:2025.11.2 dump-config` bazując na dokumentacji [11]. Zwiększyłem dostępną pamięć i wyłączyłem telemetrię dostosowując plik `neo4j.conf`.

Przeczytałem, że domyślnie format pliku konfiguracyjnego to *ISO-8859-1*, co może powodować problemy, gdybym używał polskich znaków. Próbowałem zmienić go na *UTF-8*



Rysunek 2: Podgląd danych Game of Graphs. Źródło: Repozytorium Game of Graphs [1]

ustawiając zmienną środowiskową [12].

Jednak po uruchomieniu kontenera Neo4j wyświetlał błąd konfiguracji, więc zrezygnowałem z tego ustawienia.

Do wybrania wartości ustawień dostępnej pamięci użyłem narzędzia `bin/neo4j-admin server memory-recommendation` [13] w kontenerze podając dostępną pamięć jako 4 GB.

Skonfigurowałem również automatyczną instalację pluginu *APOC Core* oraz skonfigurowałem go w pliku `apoc.conf`, zgodnie z dokumentacją [14] [15] [16].

Po uruchomieniu kontenera sprawdziłem czy Neo4j działa poprawnie, używając `cypher-shell` [17].

Możliwe jest też użycie *Aura Console* do podłączenia się do lokalnej instancji Neo4j [18], mi to jednak nie działało. Zamiast tego, żeby móc korzystać z wizualizacji uruchomiłem wbudowany interfejs webowy <http://localhost:7474/browser>.

Skonfigurowałem również rozszerzenie edytora kodu *Neo4j for VS Code*.

2.2 Wyczytanie danych

Próbowałem stworzyć bazę danych poleceniem Cypher [19] (Wykaz 1). Przy okazji chciałem też ustawić domyślną wersję Cypher na 25, żeby mieć dostęp do najnowszych funkcji języka.

Otrzymałem jednak błąd. Dopiero gdy doczytałem dokumentację zorientowałem się, że polecenie `CREATE DATABASE` nie działa w tylko w wersji *Enterprise Edition* [19].

Wykaz 1 Nieudane utworzenie bazy w Cypher

```
1 CREATE DATABASE GameOfGraphs
2 DEFAULT LANGUAGE CYPHER 25
```

Żeby utworzyć bazę danych `GameOfGraphs` zastosowałem obejście z wątku na forum [20], polegające na zmianie domyślnej bazy w pliku konfiguracyjnym `neo4j.conf`.

Przy restarcie kontenera zauważyłem ostrzeżenie, że klucz konfiguracyjny `dbms.default_database` jest przestarzały i należy użyć `initial.dbms.default_database`, więc to poprawiłem. Przy okazji skonfigurowałem też domyślną wersję Cypher na 25, ponieważ bazowo jest to już nie rozwijany Cypher 5 [21] [22].

Do importu danych chciałem użyć procedury `apoc.load.json` z pluginu *APOC*. Jednak nie działała mi ona mimo, że skonfigurowałem instalację pluginu *APOC* za pomocą zmiennej środowiskowej `NEO4J_PLUGINS`.

Po przyjrzeniu się problemowi zorientowałem się, że ponieważ używam *Docker Compose* nie mogę mieć cudzysłowów ' w okół wartości zmiennej środowiskowej `NEO4J_PLUGINS`,

tak jak jest to pokazywane w dokumentacji na przykładach [14], ponieważ przykłady te są ze składnią dla zwykłego Docker CLI.

Poprawiłem konfigurację w pliku `docker-compose.yml` i plugin *APOC* został zainstalowany poprawnie.

Do wczytania danych chciałem użyć gotowego skryptu z repozytorium *Game of Graphs* `got-import.cypher` [1]. Linter z pluginu *Neo4j for VS Code* [23] zwrócił mi uwagę na nieaktualną składnię tworzenia indeksów i constraint, więc poprawiłem ją na zgodną z dokumentacją [24] [25].

Pliki **JavaScript Object Notation (JSON)**, z których korzystał skrypt nie są już dostępne pod adresami URL w skrypcie. Udało mi się je jednak odnaleźć na branchu **release** repozytorium *Game of Graphs* [1]. Pobrałem je i zmodyfikowałem moją wersję skryptu `got-import.cypher`, żeby korzystała z lokalnych plików.

Przy próbie importu występował jednak błąd **Cannot merge the following node because of null property value for 'id': (:Person {id: null})**. Testując skrypt na mniejszym pliku, który przygotowałem, zauważyłem, że problemem jest tworzenie relacji. Ustaliłem, że problemem jest wykonanie **CASE**, który mimo danych będących **null** wykonywał kod tworzący relację do węzła, który w tym przypadku miał **id** równe **null**. Udało mi się rozwiązać ten problem stosując wersję *Generic CASE* zamiast *Simple CASE* [26].

Po tej poprawce udało mi się wczytać dane do bazy. Ich wizualizacja jest widoczna na Rysunek 3a. Na ogólnej wizualizacji widać zgrupowania postaci w danych rodach, oraz zbiór niezależnych postaci otaczający środkowe grupy.

Ponieważ bazowy zbiór *Game of Graphs* [1] zawiera tylko 4 rodzaje węzłów, wczytałem też dane o książkach z serii.

Napisałem skrypt `import_books.cypher`, który na podstawie danych z *An API of Ice And Fire* [7] tworzy węzły **Author** i **Book**, oraz relacje **WROTE**, **FOLLOWS** i **PRECEDES** między nimi.

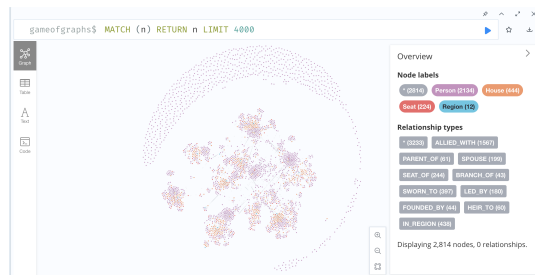
Wyliczyłem liczbę węzłów i relacji prostymi zapytaniami i przedstawiłem je w tabelach Tabela 1 i Tabela 2. Wizualizacja wczytanych danych jest widoczna na Rysunek 3b.

Tabela 1: Liczba węzłów po wczytaniu danych

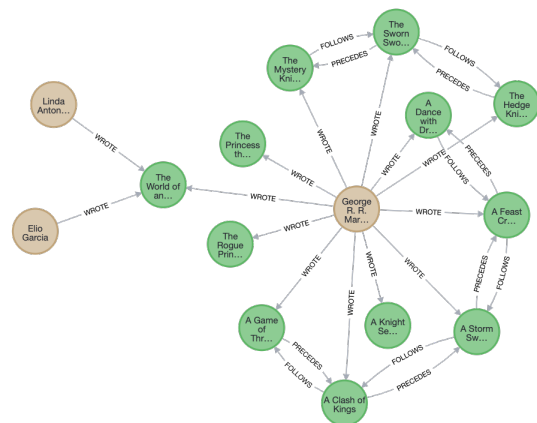
typ węzła	liczba węzłów
“Person”	2134
“House”	444
“Seat”	224
“Region”	12

Tabela 2: Liczba relacji po wczytaniu danych

relacja	typ docelowego węzła	liczba relacji
“ALLIED_WITH”	“House”	1567
“ALLIED_WITH”	“Person”	1567
“SWORN_TO”	“House”	794
“IN_REGION”	“Region”	438
“IN_REGION”	“House”	438
“SPOUSE”	“Person”	398
“SEAT_OF”	“House”	244
“SEAT_OF”	“Seat”	244
“LED_BY”	“Person”	180
“LED_BY”	“House”	180
“PARENT_OF”	“Person”	122
“BRANCH_OF”	“House”	86
“HEIR_TO”	“House”	60
“HEIR_TO”	“Person”	60
“FOUNDED_BY”	“Person”	44
“FOUNDED_BY”	“House”	44



(a) Wizualizacja bazowych danych



(b) Wizualizacja danych o książkach i autorach

Rysunek 3: Wizualizacja wczytanych danych z *Api of Ice And Fire* [7]

3 Dołączenie drugiego zbioru danych

Zdecydowałem się rozszerzyć mój zbiór danych o zbiór z informacjami o bitwach z platformy *Kaggle* [27].

3.1 Wczytanie danych o bitwach

Napisałem skrypt `import_battles.cypher`, który wczytuje dane z pliku CSV i tworzy węzły `Battle` oraz relacje z już istniejącymi węzłami `Person`, `House` i `Region` opisane w Tabeli 3.

Tabela 3: Relacje tworzone w skrypcie `import_battles.cypher`

Nazwa relacji	Węzeł źródłowy	Węzeł docelowy	Opis
TOOK_PLACE_IN	Battle	Region	Bitwa miała miejsce w danym regionie
COMMANDED_ATTACK_IN	Person	Battle	Osoba dowodzi atakiem w bitwie
COMMANDED_DEFENSE_IN	Person	Battle	Osoba dowodzi obroną w bitwie
ATTACKED_IN	House	Battle	Ród atakował w bitwie
DEFENDED_IN	House	Battle	Ród bronił się w bitwie

Do wczytania pliku [Comma-separated values \(CSV\)](#) na początku chciałem użyć procedury `apoc.load.csv`. Po pojawieniu się błędu o nieznalesionej procedurze zorientowałem się, że jest ona w pluginie *APOC Extended*, a nie w podstawowym *APOC Core* [28]. W sytuacji zdecydowałem się wypróbować domyślne polecenie `LOAD CSV`, ponieważ nie potrzebowałem żadnych specjalnych opcji dostępnych w `apoc.load.csv` [28].

Pisząc skrypt zwróciłem uwagę, żeby nie tworzyć węzłów innych niż `Battle`. Do tworzenia węzłów `Battle` użyłem `MERGE`, żeby nie tworzyć duplikatów przy ponownym uruchomieniu skryptu. Traktowałem mój główny zbiór danych jako właściwą listę postaci, domów i rodów. Żeby to osiągnąć użyłem `MATCH` zamiast `MERGE` przy wyszukiwaniu istniejących węzłów. Dodałem też walidację, korzystając z `apoc.util.validatePredicate` na wypadek gdyby nazwy różniły się między zbiorami.

Komentarz 3.1. Dane w pliku [CSV](#) posiadały niestandardowy format wymienienia wielu postaci. W kolumnach `attacker_king` i `defender_king`, w przypadku uczestnictwa kilku władców, różni królowie mieli podane najpierw imiona oddzielone /, a potem wspólne nazwisko. Przykładowo `Joffrey/Tommen Baratheon` oznaczało udział Joffrey’a Baratheona i Tommena Baratheona. Wymagało to dedykowanej logiki oddzielania postaci. W pozostałych kolumnach, z wieloma wartościami wartości, były oddzielone przecinkami.

Dostosowania wymagało też powiązanie nazw domów. W pliku [CSV](#) nazwy domów były krótkie np. `Lannister`, natomiast w bazowych danych były one pełne, np. `House Lannister of Casterly Rock`. Żeby powiązać te nazwy użyłem `CONTAINS`.

Ostatnim problemem, który napotkałem przy imporcie były różnice w nazwach postaci między dwoma zbiorami i pojawienie się nazw grup w dodatku do nazw rodów. Rozwiązałem to przygotowując mapowania nazw dowódców oraz wykluczając nazwy grup nie

będących rodami. Dodałem też logikę usuwającą przedrostek *Lord* przed wyszukaniem powiązanych postaci. Zaskoczyło mnie, że kilka nazw postaci w dodatkowym zbiorze zawiera literówki.

Po rozwiązaniu napotkanych problemów udało mi się wczytać informacje o wszystkich 38 bitwach oraz relacje (Tabela 4).

Tabela 4: Liczba relacji związanych z bitwami

relacja	typ docelowego węzła	liczba relacji
"COMMANDED_ATTACK_IN"	"Person"	130
"COMMANDED_DEFENSE_IN"	"Person"	100
"ATTACKED_IN"	"House"	79
"DEFENDED_IN"	"House"	65
"TOOK_PLACE_IN"	"Region"	38

W relacjach `COMMANDED_ATTACK_IN`, `COMMANDED_DEFENSE_IN`, `ATTACKED_IN` i `DEFENDED_IN` dodałem atrybut `won` określający czy dana strona wygrała bitwę. Chciałem też umieścić w relacjach informacje o liczebności armii, niestety jednak w danych jest tylko informacja o całkowitej liczbie wojsk, bez podziału na konkretne domy [27].

3.2 Wykorzystanie rozszerzonych danych

Byłem ciekaw jaka postać jest rodzicem najliczniejszego rodu. Żeby to zbadać napisałem zapytanie `most_descendants.cypher` korzystające z procedury `apoc.path.subgraphNodes`.

Według wyniku zapytania najwięcej potomków ma *Aegon IV* z liczbą 6 potomków. Zdziwił mnie ten wynik. Spodziewałem się znacznie większej liczby potomków nawet samego *Aegona IV*, oraz ogólnie dla przodków starych rodów. Przeanalizowałem dane prostymi zapytaniami typu Wykaz 2 wyświetlającymi relacje znanych mi postaci. Okazało się, że wiele postaci nie ma relacji do wielu swoich dzieci. Niestety wynika to z niepełności danych w zbiorze *Game of Graphs* [1].

Wykaz 2 Relacje Eddarda Starka

```

1 MATCH (p:Person)-[r]-(p2:Person)
2 WHERE p.name = "Eddard Stark"
3 RETURN p, r, p2

```

Zastanawiałem się, które bitwy mogły być najistotniejsze. Uznałem, że w najistotniejszych bitwach bierze udział wiele rodów, dowódców i żołnierzy. Na istotność bitwy

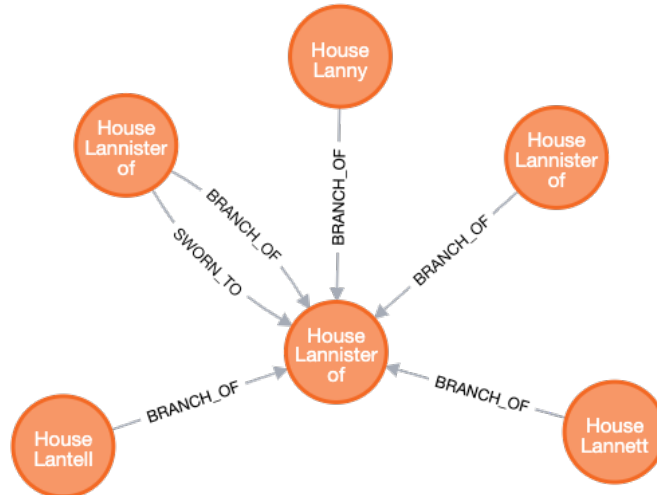
ma też wpływ czy zginął w niej lub został pojmany ktoś ważny (pola `major_death` i `major_capture`).

Uwzględniając te kryteria przygotowałem prostą heurystykę i zapytanie wyświetlające najważniejsze bitwy `most_impactfull_battles.cypher`.

Poprzednim zapytaniem znalazłem, że według mojej heurystyki najważniejszą bitwą była *Battle of Castle Black*. Chciałem zobaczyć, które postacie brały udział w tej bitwie. W tym celu napisałem zapytanie `characters_in_battle_of_castle_black.cypher`. Użyłem w nim UNION, żeby wyświetlić postaci na podstawie różnych relacji dopisując do każdego rodzaju wartości konkretne wartości kolumn.

Chciałem sprawdzić, który ród jest najbardziej agresywny. Żeby to zrobić przygotowałem zapytanie `most_aggressive_houses.cypher`, które liczy w ilu bitwach dany ród atakował. Ponieważ rody główne rody mają dodatkowe gałęzie (relacja `BRANCH_OF`), w zapytaniu przechodzę po tej relacji aż do głównego rodu i dla niego liczę ataki sumując również bitwy, w których brały udział gałęzie rodów.

Rozgałęzienia rodów można zobaczyć zapytaniem `branches_of_house.cypher` (Rysunek 4, Tabela 5).



Rysunek 4: Gałęzie rodu Lannisterów

Tabela 5: Gałęzie rodu Lannisterów

mainHouse	branchHouse
House Lannister of Casterly Rock	House Lannett
House Lannister of Casterly Rock	House Lannister of Darry
House Lannister of Casterly Rock	House Lanny
House Lannister of Casterly Rock	House Lannister of Lannisport
House Lannister of Casterly Rock	House Lantell

3.3 Ręczne tworzenie węzłów

Utworzyłem 5 nowych węzłów i powiązałem je relacjami z istniejącymi węzłami w pliku [manual_data_creation.cypher](#). Dodałem głównie dane, które były brakujące przy imporcie danych o bitwach.

Dodałem nowy rodzaj węzła **Group**, reprezentujący grupy postaci niebędące rodami. Utworzyłem grupy **Free folk** i **Night's Watch**.

Free folk powiązałem relacją **IN_REGION** z regionem **Beyond the Wall**, ponieważ są to dzikie plemiona żyjące na północ od Muru.

Grupe **Night's Watch** powiązałem relacją **IN_REGION** z regionami **The North** i **Beyond the Wall**, ponieważ ich główna siedziba to Mur, który jest granicą między tymi regionami. Często członkowie tej organizacji wyruszają na wyprawy na północ od Muru (**Beyond the Wall**). Dodałem też relację do postaci **Jon Snow**, który był członkiem Nocnej Straży (**Night's Watch**).

Utworzyłem węzły dla brakujących postaci **Styr** i **Ryman Frey** oraz przypisałem im relacje **ALLIED_WITH** i **MEMBER_OF** do odpowiednich domów i grup.

Żeby powiązać nowe węzły z danymi o bitwach wyszukałem po **battleNumber** bitwy i dodałem odpowiednie relacje.

4 Optymalizacja zapytań

Optymalizacja napisanych wcześniej zapytań i analiza ich wydajności.

4.1 Indeksy

Indeksy w Neo4j tworzą dodatkową kopię danych, która pozwala na szybsze wyszukiwanie węzłów lub krawędzi według zadanych wartości. [29] Domyślnie Neo4j tworzy indeksy typu B-tree [30], jednak dostępne są też inne typy indeksów, takie jak **POINT INDEX** dla danych geoprzestrzennych czy dedykowane indeksy do typów tekstowych **TEXT INDEX** oraz indeksy pełnotekstowe. [31]. Domyślnie dostępne są też indeksy wektorowe.

Ponieważ indeksy duplikują dane spowalnia to zapisy sprawia, że baza zajmuje więcej miejsca. Należy ostrożnie tworzyć indeksy i regularnie analizować czy ciągle są przydatne. [32].

Już po utworzeniu czystej bazy w Neo4j istnieją w niej dwa indeksy token lookup. Służą one do wyszukiwania po etykiecie. Jeden z jest dla węzłów, a drugi dla relacji.

Na początku zdziwiłem się, gdy je zobaczyłem uruchamiając polecenie **SHOW INDEXES**. Potem przeczytałem, że jest to domyślne zachowanie Neo4j [31].

Podstawową techniką optymalizacji jest dodanie indeksów do właściwości używanych w zapytaniach.

W zapytaniach `branches_of_house.cypher` i `characters_in_battle_of_castle_black.cypher` stosuję filtrowanie po właściwości.

Mogę zoptymalizować te zapytania dodając indeksy do właściwości `name` węzłów `House` i `Battle`. Takie indeksy istniały już w podstawowym skrypcie importu danych w repozytorium *Game of Graphs* [1].

Przetestowałem działanie indeksu w zapytaniu `branches_of_house.cypher`. Skorzystałem w tym celu z polecenia `EXPLAIN`, wykonując zapytanie z usuniętym indeksem i działającym.

Żeby uniknąć wpływu cache'u na wyniki testu, pomiędzy testami usuwałem całkowicie kontener Neo4j. Zostawiałem oczywiście wolumen z danymi.

Porównanie statystyk wykonania i planów zapytania (Rysunek 5) pokazuje, że włączenie indeksu prowadzi do zaczęcia wykonania zapytania przez `NodeIndexSeek` zamiast `NodeByLabelScan`. Dzięki temu liczba przeczytanych węzłów spadła z 910 do 23, a czas wykonania z 194 ms do 93 ms. Jest to znacząca poprawa wydajności.

Operator	Id	Details	Estimated Rows	Rows	DB Hits	Memory (Bytes)	Page Cache Hits/Misses
+ProduceResults	0	mainHouse, branchHouse	22	5	0	0	0/0
+Projection	1	cache[mainHouse.name] AS mainHouse, cache[branch.name] AS branchHouse	22	5	0		0/0
+Apply	2		22	5	0		0/0
+Optional	3	mainHouse	22	5	0		0/0
+CacheProperties	4	cache[branch.name]	2	5	5		0/0
+Filter	5	branch:House	2	5	5		0/0
+VarLengthExpand(All)	6	(mainHouse)-[:BRANCH_OF*]-(branch)	2	5	11	384	0/0
+Argument	7	mainHouse	22	1	0		0/0
+Filter	8	cache[mainHouse.name] = \$autostring_0	22	1	444		0/0
+NodeByLabelScan	9	mainHouse:House	444	444	445		0/0

Total database accesses: 910, total allocated memory: 464

5 rows

ready to start consuming query after 93 ms, results consumed after another 26 ms

(a) Plan zapytania bez indeksu

Operator	Id	Details	Estimated Rows	Rows	DB Hits	Memory (Bytes)	Page Cache Hits/Misses	Indexes Used
+ProduceResults	0	mainHouse, branchHouse	1	5	0	0	0/0	
+Projection	1	cache[mainHouse.name] AS mainHouse, cache[branch.name] AS branchHouse	1	5	0		0/0	
+Apply	2		1	5	0		0/0	
+Optional	3	mainHouse	1	5	0		0/0	
+CacheProperties	4	cache[branch.name]	0	5	5		0/0	
+Filter	5	branch:House	0	5	5		0/0	
+VarLengthExpand(All)	6	(mainHouse)-[:BRANCH_OF*]-(branch)	0	5	11	384	0/0	
+Argument	7	mainHouse	1	1	0		0/0	
+NodeIndexSeek	8	RANGE INDEX mainHouse:House(name) WHERE name = \$autostring_0, cache[mainHouse.name]	1	1	2		0/0	house_name_index: 1

Total database accesses: 23, total allocated memory: 464

5 rows

ready to start consuming query after 194 ms, results consumed after another 18 ms

(b) Plan zapytania z indeksem

Rysunek 5: Zapytanie o gałęzie rodu Lannisterów z i bez indeksu

Często zdarza mi się robić literówki w nazwach postaci. Chciałem wypróbować czy indeks `FULLTEXT` pomoże mi znajdować właściwe postaci mimo literówek. W dokumentacji zauważyłem, że indeks tego typu pozwala na luźne dopasowanie i zwrócenie poza wynikiem współczynnika dopasowania [33]. Przetestowałem indeks `FULLTEXT` w zapytaniu `fulltext_character_search.cypher`.

Niestety w przypadku literówki w imieniu postaci *Arja Stark* (prawidłowo *Arya Stark*)

indeks zwracał mi wszystkie postaci o nazwisku *Stark* z tym samym `score`, co nie rozwiązywało mojego problemu.

Świadczy to o tym, że indeks `FULLTEXT` oddzielnie tonizuje każde słowo, co potwierdza dokumentacja [33]. Zainteresowało mnie, że indeks `FULLTEXT` ma możliwość aktualizacji w tle, dzięki czemu nie spowalnia on bezpośrednio zapisu danych, gdy ustawi się opcję `fulltext.eventually_consistent` [33].

4.2 Optymalizacja struktury grafu

Inną, niż indeksy, techniką optymalizacji zapytań jest przebudowanie grafu, by zwiększyć wydajność zapytań. W tym celu można na przykład dodać relacje skracające ścieżki między węzłami.

W moim zapytaniu `most_aggressive_houses.cypher` przechodzę po wielu poziomach relacji `BRANCH_OF`, żeby znaleźć główny ród. Ponieważ gałęzie domów mogą mieć swoje podgałęzie, które dalej mogą być rozgałęzione, zapytanie nie korzysta z kryterium lokalności i może eksplodować na dużą część grafu.

Żeby to zoptymalizować, mogę dodać relacje `MAIN_HOUSE` bezpośrednio łączące gałęzie domów z ich głównymi rodami. Utworzyłem tę relację i zaktualizowałem zapytanie w skrypcie `optimized_most_aggressive_houses.cypher`.

5 Dane przestrzenne

Funkcje przestrzenne pozwalają na wyszukiwanie nie tylko na podstawie relacji między węzłami grafu, ale też na podstawie dwu, lub rzy wymiarowego położenia w systemie kartezjańskim lub WGS 84 [34]. Możliwe jest też korzystanie z danych przestrzennych w połączeniu z klasycznymi algorytmami przeszukiwania grafów, takimi jak algorytm Dijkstry [35].

Mój zbiór danych ani rozszerzenie nie zawierają danych przestrzennych. Chciałem dodać dane o lokalizacji do bitew. Żeby dane mogły mieć podstawę w geografii świata powieści, poprosiłem model [Artificial Intelligence \(AI\)](#) o utworzenie pliku rozwijającego moje dane o bitwach o współrzędne geograficzne w [konwersacji](#).

Nie udało mi się pobrać pliku `CSV`, ale otrzymałem skrypt do przygotowania danych. Lekko go zmieniłem, ponieważ uznałem, że wystarczą mi tylko kolumny `name`, `latitude` i `longitude`. Dostosowałem też ścieżki. Skrypt zapisałem jako `add_coordinates.py` i uruchomiłem generując plik `CSV` z danymi przestrzennymi.

Kontując [konwersację z modelem AI](#), wygenerowałem dane przestrzenne dla siedzib rodów (*Seat*).

Wygenerowane dane przestrzenne dodałem do węzłów skryptem `add_spatial_data.cypher`.

Zastanawiało mnie czy w okolicy siedzib danych rodów było średniej więcej bitew niż w innych. Przygotowałem zapytanie `battles_near_house_seats.cypher`, które znajduje bitwy w zadanym promieniu od siedzib rodów i agreguje ich średnią oraz najbliższą odległość.

Wcześniej analizowałem dane związane z bitwą *Battle of Castle Black*. Uznałem, że warto sprawdzić czy rody z okolicznymi siedzibami brały udział w tej bitwie. Napisałem w tym celu zapytanie `house_seats_near_battle.cypher`. Wykorzystałem w nim funkcję odległości `point.distance` [34].

Żeby zobaczyć wszystkie bitwy, które miały miejsce na Północy (*The North*), eksperymentalnie wyznaczyłem przybliżony prostokąt ograniczający ten region. Następnie korzystając z funkcji przestrzennej `point.withinBBBox()` [34] wylistowałem chronologicznie bitwy w tym obszarze w zapytaniu `battles_in_bounding_box.cypher`.

Wydażność zapytań przestrzennych może stanowić problem, szczególnie jeśli, tak jak w przypadku moich zapytań, nie występuje relacja między różnymi rodzajami węzłów, które są używane w zapytaniu. Prowadzi to do konieczności przeszukania wszystkich możliwych kombinacji węzłów, co dla dużej bazy byłoby bardzo wolne.

Pomocne w rozwiązaniu problemów z wydajnością mogą być indeksy `POINT` [31] i uwzględnienie relacji w strukturze grafu, które pozwolą ograniczyć przeszukiwane węzły.

Do zalet zapytań przestrzennych należy łatwość ich stosowania. W przeciwieństwie do poprzednich *System zarządzania bazami danych (SZBD)*, które badałem, Neo4j domyślnie wspiera zapytania tego typu nawet bez *APOC*.

Sądzę, że wadą może być to, że łatwo napisać zapytanie przestrzenne, które będzie mało wydajne, przez to, że będzie wymagało analizy wszystkich węzłów danego typu. Da się temu zapobiegać przez odpowiednią strukturę grafu, ale nie jest to łatwe.

Za zapytanie przestrzenne może być też uznane wyszukiwanie najkrótszej ścieżki po węzłach grafu. Przetestowałem zastosowanie algorytmu Dijkstry do tego w pliku `shortest_path.cypher`.

6 Procedury

Procedury w Neo4j pozwalają na wykonywanie w ramach zapytań Cypher kodu napisanego w Javie i prawdopodobnie w innych językach *Java Virtual Machine (JVM)*. Dzięki temu można rozszerzyć możliwości zapytań o logikę, która byłaby trudna lub niemożliwa do wyrażenia w Cypher [36].

W porównaniu do procedur w bazach relacyjnych nie są pisane w języku zapytań, a w innym języku programowania. Dostarcza to zupełnie innych możliwości, które wykraczają poza podstawowy język bazy.

Podoba mi się to podejście, ponieważ czuję się znacznie biegłej w języku Java niż w SQL czy Cypher.

Przy imporcie danych bitew imiona królów były czasem w nietypowym formacie (Remark 3.1). Wymagało to napisania dedykowanej logiki parsowania w Cypher. Zdecydowanie łatwiej jest mi wyrazić taką logikę w Javie. Chcę też ukryć złożoność tej logiki w moim kodzie importu.

Z tych powodów przygotowałem moją własną procedurę `ParseKingNames` która realizuje tę logikę.

Projekt Java z kodem procedury bazowałem na oficjalnym szablonie [37], do którego dodałem moją procedurę. Żeby wczytać procedurę do bazy należy skompilować projekt do `Java ARchive (JAR)` i umieścić go w katalogu `/plugins` [14]. Zrealizowałem to tworząc dedykowany obraz Docker w pliku `Dockerfile`. Do uruchomienia kodu z projektu wymagane było też dodanie przestrzeni nazw w pliku konfiguracyjnym `neo4j.conf`.

Po przygotowaniu procedury i wgraniu jej do bazy przetestowałem jej działanie w pliku `parse_king_names.cypher`.

7 Analiza produkcyjna

Uznałem, że dobrym sposobem na podział bazy na kilka maszyn jest wyznaczenie podgrafów spójnych. Rozdzielenie grafu spójnego może prowadzić do tego, że niektóre zapytania będą wymagały połączeń między maszynami, co na pewno wpłynie na wydajność.

Żeby znaleźć spójne podgrafy skorzystałem z algorytmu *Weakly Connected Components* z biblioteki *Graph Data Science* [38], kierując się poradami z wątku na forum [39].

Dodałem plugin *Graph Data Science* do Neo4j korzystając ze zmiennej środowiskowej `NEO4J_PLUGINS` w pliku `docker-compose.yml` i mechanizmu automatycznej instalacji [14].

Skryptami w pliku `disconnected_subgraphs.cypher` ustaliłem, że w mojej bazie są aktualnie 4 rodzaje spójnych podgrafów przedstawionych w Tabeli 6.

Tabela 6: Statystyki spójnych podgrafów w bazie

Rodzaj węzłów	Liczba spójnych podgrafów	Średnia wielkość	Odchylenie std.	Min	Max
Person	692	1.02	0.21	1	5
Person, House, Seat, Region, Battle, Group	1	2150	0	2150	2150
Person, House	1	2	0	2	2
Book, Author	1	15	0	15	15

Z mojej analizy wynika, że informacje o książkach i autorach nie są powiązane z resztą grafu. Dane te więc można łatwo umieścić na oddzielnej maszynie.

Charakterystyczny jest też jeden duży spójny podgraf zawierający większość danych o postaciach, rodach i bitwach. Podział tego podgrafu będzie wymagał dalszej analizy.

Zauważyłem też wiele podgrafów składających się jedynie z postaci. Dodatkowo często postaci te nie mają relacji do innych węzłów (średnia wielkość: 1,02 węzła). Tego typu dane można też łatwo rozdzielić między maszynami.

Podejrzewam, że nawet w znacznie większym zbiorze danych występowałyby takie same trendy.

W podobny sposób odnalazłem mosty w mojej bazie w skrypcie `bridges.cypher`. Miałem nadzieję znaleźć mosty łączące duże części grafu. Tego typu luźno połączone części grafu byłyby dobrą podstawą do podziału danych między maszyny.

Posortowałem mosty według wielkości mniejszego podgrafu po usunięciu mostu. Niestety nawet w najlepszym przypadku był to podział podgrafu na podgrafy o 2143 i 7 węzłach. Być może w danych produkcyjnych podgrafy po usunięciu mostów byłyby większe, ale dla aktualnych wyników nie jest to dobry kandydat na podział bazy.

Przeanalizowałem też węzły przegubowe w skrypcie `articulation_points.cypher`. Węzły o najwyższym stopniu (liczbie relacji) to największe rody. Dają one potencjał na rozdzielenie bazy na maszyny lub kopie danych do odczuty skupione w ramach jednego dużego rodu. Możliwe, że spora część zapytań będzie dotyczyła jednego rodu, ale należałoby to zweryfikować.

8 APOC

Już przy instalacji i konfiguracji (Sekcja 2.1) Neo4j natrafiłem na bibliotekę *APOC*. Ponieważ wydała się bardzo przydatna korzystałem z niej już w skryptach.

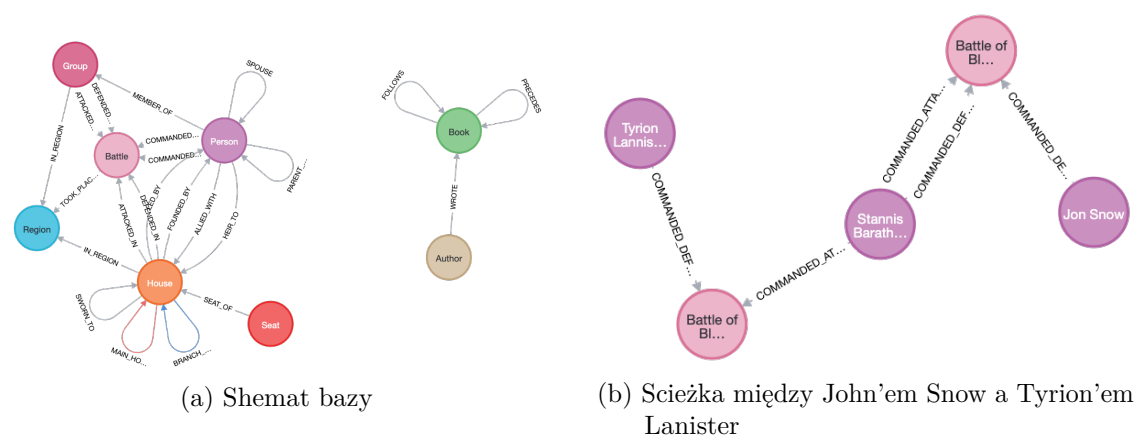
- `apoc.load.jsonArray` w `got-import.cypher` i `import_books.cypher`,
- `apoc.convert.toMap` w `got-import.cypher` i `import_books.cypher`,
- `apoc.map.clean` i `apoc.map.fromPairs` w `got-import.cypher` i `import_books.cypher`,
- `apoc.util.validatePredicate` w `import_battles.cypher`,
- `apoc.path.subgraphNodes` w `most_descendants.cypher`,
- `apoc.load.json` w `playground.cypher`.

Procedury z *APOC* były kluczowe nawet do podstawowych operacji, takich jak wczytanie danych z plików `JSON`. Co ciekawe wczytywanie danych z plików `CSV` jest dostępne domyślnie, jednak wydaje mi się, że stosunkowo niedawno zostało rozbudowane o przydatne funkcje, ponieważ dokumentacja `apoc.load.csv` wspomina o brakujących możliwościach `LOAD CSV`, które jednak w większości wyglądają aktualnie na dostępne [28] [40].

Wypróbowałem też kilka innych procedur z *APOC*, które wydały mi się ciekawe.

- `apoc.cypher.run` w `dynamic_cypher.cypher` do dynamicznie tworzonych i parametryzowanych zapytań,
- `apoc.text.jaroWinklerDistance` w `fuzzy_search.cypher` do wyszukiwania postaci z podobnymi nazwami pomimo literówek. Próbowałem to osiągnąć wcześniej z indeksem pełnotekstowym, jednak dopiero to podejście zadziałało,
- `apoc.meta.graph` i `apoc.meta.stats` w `graph_schema.cypher` do analizy schematu i statystyk grafu,
- `apoc.path.subgraphNodes` w `nodes_within_distance.cypher` do znajdowania węzłów w określonej odległości od danego węzła,
- `apoc.algo.dijkstra` w `shortest_path.cypher` do znalezienia ścieżki między węzłami.

Wizualizacje ciekawych wyników ilustruję na Rysunek 6.



Rysunek 6: Wizualizacje wybranych ciekawych procedur *APOC*

9 Wnioski

Bazy grafowe są zdecydowanie różne od innych rodzajów baz, z którymi miałem do tej pory styczność. Pozwalają na znacznie bardziej naturalne/intuicyjne oddanie zależności i informacji z rzeczywistego świata. Sądzę, że przedstawianie danych w postaci węzłów i relacji jest znacznie bardziej intuicyjne model relacyjny czy dokumentowy.

Struktura grafu ma bardzo istotny wpływ na wydajność zapytań. Myślę, że w bazach grafowych szczególnie ważne jest optymalizowanie struktury bazy do zadanego problemu. Sądzę, że bardzo łatwo jest spowodować, że baza grafowa będzie bardzo wolna przez niedobrze przemyślaną strukturę i zapytania.

Znajomość i biegłość z algorytmów grafowych i matematycznej teorii grafów na pewno jest sporym atutem w pracy z bazami tego typu.

Wizualizacje danych przychodzą naturalnie w bazie grafowej i pozwalają na skuteczne analizowanie niektórych problemów.

Ogólnie uważam, że bazy grafowe są bardzo potężnym narzędziem, które nadaje się do rozwiązywania konkretnych problemów. Ich skuteczna obsługa wymaga jednak sporej wiedzy i przemyślanego podejścia.

Źródła

1. neo4j-examples/game-of-thrones: Neo4j Database of A Song of Ice and Fire - Game of Thrones. <https://github.com/neo4j-examples/game-of-thrones?tab=readme-ov-file>. Accessed 28 grudz 2025
2. Martin GRR (1996) A Game of Thrones. Bantam Books
3. Martin GRR (1998) A Clash of Kings. Bantam Books
4. Martin GRR (2000) A Storm of Swords. Bantam Books
5. Martin GRR (2005) A Feast for Crows. Bantam Books
6. Martin GRR (2011) A Dance with Dragons. Bantam Books
7. joakimskoog/AnApiOfIceAndFire: An API of Ice And Fire is the world's greatest source for quantified and structured data from the universe of Ice and Fire (as well as the HBO series Game of Thrones). <https://github.com/joakimskoog/AnApiOfIceAndFire>. Accessed 28 grudz 2025
8. Introduction - Operations Manual. https://neo4j.com/docs/operations-manual/current/introduction/#_neo4j_editions. Accessed 3 sty 2026
9. Getting started with Neo4j in Docker - Operations Manual. <https://neo4j.com/docs/operations-manual/current/docker/introduction/>. Accessed 3 sty 2026
10. Persisting data with Docker volumes - Operations Manual. <https://neo4j.com/docs/operations-manual/current/docker/mounting-volumes/>. Accessed 3 sty 2026
11. Modify the default configuration - Operations Manual. <https://neo4j.com/docs/operations-manual/current/docker/configuration/>. Accessed 3 sty 2026
12. The neo4j.conf file - Operations Manual. <https://neo4j.com/docs/operations-manual/current/configuration/neo4j-conf/>. Accessed 4 sty 2026

13. Get initial memory recommendations - Operations Manual. <https://neo4j.com/docs/operations-manual/current/configuration/neo4j-admin-memrec/>. Accessed 4 sty 2026
14. Plugins - Operations Manual. <https://neo4j.com/docs/operations-manual/current/docker/plugins/>. Accessed 3 sty 2026
15. Installation - APOC Core Documentation. <https://neo4j.com/docs/apoc/current/installation/#docker>. Accessed 3 sty 2026
16. Configuration Options - APOC Core Documentation. <https://neo4j.com/docs/apoc/current/config>. Accessed 3 sty 2026
17. Cypher Shell - Operations Manual. <https://neo4j.com/docs/operations-manual/current/cypher-shell/#cypher-shell-standalone>. Accessed 4 sty 2026
18. Access Neo4j - Operations Manual. <https://neo4j.com/docs/operations-manual/current/docker/accessing-neo4j/>. Accessed 4 sty 2026
19. Create databases - Operations Manual. <https://neo4j.com/docs/operations-manual/current/database-administration/standard-databases/create-databases>. Accessed 4 sty 2026
20. (2020) Error occurs when creating a new database under Neo4j 4.0 - Stack Overflow. <https://stackoverflow.com/questions/60429947/error-occurs-when-creating-a-new-database-under-neo4j-4-0>. Accessed 4 sty 2026
21. Configuration settings - Operations Manual. https://neo4j.com/docs/operations-manual/current/configuration/configuration-settings/#config_db.query.default_language. Accessed 4 sty 2026
22. Select Cypher version - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/queries/select-version/>. Accessed 4 sty 2026
23. Neo4j for VS Code - Visual Studio Marketplace. <https://marketplace.visualstudio.com/items?itemName=neo4j-extensions.neo4j-for-vscode>. Accessed 5 sty 2026
24. Create, show, and drop constraints - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/constraints/managing-constraints>. Accessed 5 sty 2026
25. Syntax - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/indexes/syntax/>. Accessed 5 sty 2026

26. Conditional expressions (CASE) - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/expressions/conditional-expressions/#case-generic>. Accessed 5 sty 2026
27. Game of Thrones. <https://www.kaggle.com/datasets/mylesoneill/game-of-thrones>. Accessed 7 sty 2026
28. Load CSV - APOC Extended Documentation. <https://neo4j.com/labs/apoc/5/import/load-csv/>. Accessed 8 sty 2026
29. Indexes - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/indexes/>. Accessed 10 sty 2026
30. (2021) Neo4j Performance Architecture Explained & 6 Tuning Tips. <https://graphable.ai/blog/neo4j-performance/>. Accessed 10 sty 2026
31. Create, show, and drop indexes - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/indexes/search-performance-indexes/managing-indexes>. Accessed 10 sty 2026
32. Query tuning - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/planning-and-tuning/query-tuning/>. Accessed 10 sty 2026
33. Full-text indexes - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/full-text-indexes/#query-full-text-indexes>. Accessed 10 sty 2026
34. Spatial functions - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/functions/spatial/#functions-withinBBox>. Accessed 11 sty 2026
35. Lyon W (2023) Spatial Search Functionality With Neo4j - William Lyon. <https://lyonwj.com/blog/making-sense-of-geospatial-data-with-knowledge-graphs-neo4j>. Accessed 11 sty 2026
36. User-defined procedures - Java Reference. <https://neo4j.com/docs/java-reference/current/extending-neo4j/procedures/#call-procedure>. Accessed 11 sty 2026
37. neo4j-examples/neo4j-procedure-template: Example code for building User Defined Procedures and Functions for Neo4j. <https://github.com/neo4j-examples/neo4j-procedure-template#>. Accessed 11 sty 2026

38. Weakly Connected Components - Neo4j Graph Data Science. <https://neo4j.com/docs/graph-data-science/current/algorithms/wcc/#algorithms-wcc-examples-write>. Accessed 11 sty 2026
39. (2020) Finding all disconnected subgraphs in a graph - Neo4j Graph Platform / Graph Data Science / Graph Analytics - Neo4j Online Community. <https://community.neo4j.com/t/finding-all-disconnected-subgraphs-in-a-graph/27241>. Accessed 11 sty 2026
40. LOAD CSV - Cypher Manual. <https://neo4j.com/docs/cypher-manual/current/clauses/load-csv/>. Accessed 11 sty 2026

Zastosowane skróty

AI Artificial Intelligence

CSV Comma-separated values

JAR Java ARchive

JSON JavaScript Object Notation

JVM Java Virtual Machine

SZBD System zarządzania bazami danych